Sistema de Priorización de Atención: Diseño e Implementación Inicial de una Cola de Prioridad por Niveles y una Pila sobre Listas Enlazadas

Johan Adrian Torres Robles, Esteban Mora Lopez, Miguel Angel Rodriguez Bohorquez
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{jotorresro, emoral, mrodriguezbo}@unal.edu.co

***Resumen*—Los centros de atención que reciben solicitudes de servicio deben atenderlas según su nivel de urgencia y no según su orden de llegada, además de poder consultar y cancelar solicitudes puntuales y revisar las atenciones realizadas. Este documento presenta el diseño e implementación inicial de un Sistema de Priorización de Atención (SPA) que resuelve este problema con estructuras lineales. Una cola de prioridad formada por una cola FIFO por nivel de urgencia, cada una sobre una lista enlazada con referencias a la cabeza y a la cola, registra y atiende solicitudes en tiempo constante y resuelve el desempate por orden de llegada sin comparaciones adicionales; la búsqueda y la cancelación por identificador recorren estas listas en tiempo lineal; y una pila enlazada registra el historial de atenciones. La primera entrega incluye una implementación funcional con menú de consola que cubre el registro, la atención, la búsqueda, la cancelación y la consulta del historial, verificada mediante un programa de prueba manual con casos borde. Deshacer la última atención y la reducción del costo de la búsqueda quedan planteados para las siguientes entregas.**

***Index Terms*—cola de prioridad, cola, lista enlazada, pila, estructuras de datos**

I. Introducción

Los centros de atención que reciben solicitudes de servicio (salas de urgencias, mesas de ayuda técnica, módulos de atención al cliente) enfrentan un mismo reto operativo: las solicitudes no deben atenderse en el orden en que llegan, sino según su nivel de urgencia, y el personal necesita poder consultar o cancelar solicitudes específicas y revisar las atenciones realizadas sin tener que reorganizar manualmente toda la fila.

Resolver este problema de manera eficiente requiere estructuras de datos que soporten, simultáneamente: (i) obtener en todo momento la solicitud más urgente sin tener que ordenar repetidamente toda la colección, (ii) respetar el orden de llegada entre solicitudes de igual urgencia, (iii) ubicar y retirar una solicitud puntual por su identificador, y (iv) registrar las atenciones en el orden en que ocurren para poder consultarlas y, en una entrega posterior, revertir la más reciente.

Este documento presenta el diseño e implementación inicial de un *Sistema de Priorización de Atención* (SPA), que resuelve el problema anterior combinando estructuras lineales estudiadas en la asignatura: colas y listas enlazadas para el orden de atención, y una pila para el historial de atenciones.

La contribución de esta primera entrega consiste en: (a) la formulación del problema y su alcance, (b) la selección y justificación técnica de las estructuras de datos empleadas, y (c) una implementación funcional que permite registrar, atender, buscar y cancelar solicitudes, y consultar el historial de atenciones.

El resto del documento se organiza así: la Sección II presenta trabajo relacionado; la Sección III describe los objetivos, el alcance, los requerimientos y el diseño de la solución, incluyendo las estructuras de datos seleccionadas; la Sección IV reporta el estado actual de la implementación; la Sección V discute las limitaciones conocidas; y la Sección VI resume el estado del proyecto y el trabajo futuro.

II. Trabajo relacionado

El problema de atender elementos según su prioridad se modela con el tipo abstracto *cola de prioridad*, cuyas operaciones básicas son insertar un elemento y extraer el de mayor prioridad [1], [2]. La literatura distingue entre el tipo abstracto y su representación: el mismo comportamiento puede implementarse con estructuras distintas, y es la representación la que determina el costo de cada operación.

Una representación directa es una lista ordenada por prioridad, que permite extraer el elemento más prioritario en tiempo constante, pero cuya inserción cuesta $O(n)$, pues debe recorrer la lista hasta encontrar la posición correcta. En el extremo opuesto, una lista sin orden inserta en tiempo constante pero obliga a recorrerla completa, en $O(n)$, cada vez que se extrae el elemento más prioritario [2]. Para prioridades arbitrarias, la representación clásica que equilibra ambos costos es el *heap* binario, con inserción y extracción en $O(\log n)$ [1].

Cuando la prioridad solo puede tomar un número pequeño y fijo de valores, como ocurre en los esquemas de triage que clasifican a cada paciente en unos pocos niveles de

urgencia, existe una alternativa más simple: mantener una cola FIFO independiente por cada nivel. Insertar consiste en encolar en el nivel correspondiente, y extraer consiste en revisar los niveles en orden de urgencia hasta encontrar uno no vacío. Como el número de niveles es una constante, ambas operaciones cuestan $O(1)$, y el orden FIFO de cada cola resuelve de forma natural el desempate por orden de llegada. Esta es la representación que adopta este proyecto (Sección III).

Sistemas reales de mesa de ayuda y de triage hospitalario resuelven un problema equivalente: priorizar solicitudes por urgencia y permitir su consulta puntual. Este proyecto no pretende replicar un sistema de producción, sino aislar y resolver correctamente el núcleo algorítmico común a este tipo de sistemas.

III. Metodología

III-A. Objetivos

Objetivo general. Diseñar e implementar un sistema que permita registrar, priorizar, atender, cancelar y consultar solicitudes de atención de forma eficiente, aplicando estructuras de datos apropiadas para cada operación.

Objetivos específicos.

1. Implementar una cola de prioridad formada por una cola FIFO por nivel de urgencia, cada una sobre una lista enlazada, para determinar la solicitud a atender en tiempo constante.
2. Implementar la búsqueda y la cancelación de una solicitud pendiente por su identificador.
3. Implementar una pila sobre lista enlazada que registre el historial de atenciones y permita, en una entrega posterior, deshacer la más reciente.
4. Integrar las estructuras en una interfaz de consola que exponga las operaciones del sistema.

III-B. Alcance y limitaciones

El sistema completo contempla: registrar una solicitud con un identificador, un nombre y un nivel de urgencia (de 1, más urgente, a 5, menos urgente); atender la solicitud de mayor prioridad; buscar y cancelar una solicitud pendiente por identificador; consultar el historial de atenciones; y deshacer la atención más reciente. Esta primera entrega implementa todas estas operaciones excepto deshacer, que queda diseñada y se implementará en la siguiente entrega.

Quedan explícitamente fuera del alcance del proyecto: persistencia en base de datos, interfaz gráfica, acceso concurrente de múltiples usuarios y autenticación. Se asume que los identificadores son proporcionados por quien registra la solicitud; el sistema rechaza un identificador que ya pertenezca a otra solicitud pendiente. Se asume también que la urgencia toma solo cinco valores, supuesto del que depende el costo constante de la cola de prioridad elegida.

III-C. Requerimientos principales

- RF1: El sistema debe permitir registrar una solicitud con identificador único entre las pendientes, nombre y nivel de urgencia entre 1 y 5, rechazando valores fuera de ese rango.
- RF2: El sistema debe atender siempre la solicitud pendiente de mayor urgencia; en caso de empate, debe atenderse primero la que llegó antes.
- RF3: El sistema debe permitir buscar una solicitud pendiente por su identificador.
- RF4: El sistema debe permitir cancelar una solicitud pendiente de modo que nunca sea atendida.
- RF5: El sistema debe permitir deshacer la atención más reciente, devolviendo la solicitud correspondiente a la cola de pendientes.
- RF6: El sistema debe permitir consultar el historial de solicitudes atendidas.

III-D. Diseño de la solución

El sistema se organiza en los siguientes componentes: una clase `Ticket` que representa una solicitud; una cola genérica `LinkedQueue<T>` sobre lista enlazada; una cola de prioridad `LevelPriorityQueue`, formada por un arreglo de cinco `LinkedQueue<Ticket>`, una por nivel de urgencia; una pila genérica `LinkedStack<T>`; y un historial `ServiceHistory` construido sobre esa pila. Una clase fachada, `TriageSystem`, coordina la cola de prioridad y el historial para que cada operación pública los mantenga consistentes, y una clase `Main` ofrece el menú de consola.

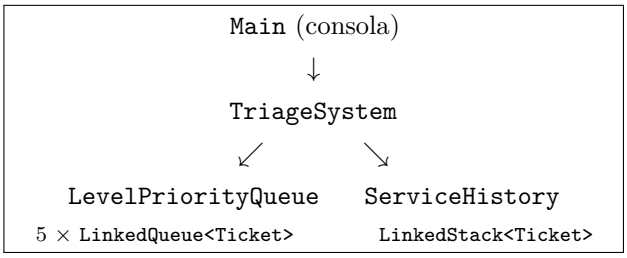


Figura 1. Componentes principales del sistema y la estructura de datos que resuelve cada responsabilidad.

III-E. Estructuras de datos seleccionadas

Cola de prioridad por niveles. Representa el conjunto de solicitudes pendientes. Está formada por un arreglo de cinco colas FIFO; la cola de índice $u - 1$ contiene, en orden de llegada, las solicitudes de urgencia u. Debe soportar insertar una solicitud y extraer la más prioritaria. Insertar consiste en encolar al final de la cola de su nivel, y extraer consiste en recorrer los niveles desde el más urgente y desencolar del primero que no esté vacío (Algoritmo 1). Como el número de niveles es la constante $k = 5$, ambas operaciones cuestan $O(k) = O(1)$. El desempate por orden de llegada (RF2) no requiere comparaciones adicionales: lo garantiza el comportamiento FIFO de cada cola. Se descartó una lista ordenada por prioridad, cuya inserción

Algoritmo 1 Atender la siguiente solicitud

```
1: function ServeNext
2:     for u ← 1 to 5 do
3:         if L[u] no está vacía then
4:             t ← Dequeue(L[u])
5:             apilar t en S
6:             return t
7:         end if
8:     end for
9:     return NULL
10: end function
```

cuesta $O(n)$, y no se requiere un *heap*, cuyo costo $O(\log n)$ solo se justifica cuando la prioridad puede tomar muchos valores distintos.

Cola sobre lista enlazada con referencias a cabeza y cola. Cada nivel es una cola implementada como lista simplemente enlazada que mantiene una referencia a la cabeza (frente) y otra al último nodo (final). La referencia al final es la que permite encolar en $O(1)$; sin ella, encolar requeriría recorrer la lista en $O(n)$. Se prefirió la lista enlazada sobre un arreglo circular porque el número de solicitudes por nivel no se conoce de antemano y porque la cancelación necesita retirar un elemento arbitrario, operación que en una lista enlazada solo exige reenlazar el nodo anterior con el siguiente, sin desplazar elementos.

Búsqueda y cancelación por identificador. La cola de prioridad resuelve eficientemente "¿cuál es la siguiente solicitud a atender?", pero no "¿dónde está la solicitud con identificador X?". En esta entrega la búsqueda recorre las colas de los cinco niveles, con costo $O(n)$ en el peor caso. La cancelación usa esa búsqueda y luego elimina el nodo de la lista enlazada de su nivel en $O(1)$, actualizando la referencia al final cuando el nodo eliminado es el último (Algoritmo 2); su costo total es $O(n)$. La misma búsqueda permite rechazar identificadores repetidos al registrar, por lo que el registro completo cuesta $O(n)$. Reducir estos costos a $O(1)$ esperado mediante un índice por identificador es una de las mejoras previstas para las siguientes entregas (Sección V).

Pila (lista enlazada simple, encabezado único). Registra las solicitudes ya atendidas en orden inverso de atención. Se eligió una pila porque la operación prevista, deshacer la atención más reciente (RF5), es exactamente el comportamiento LIFO: apilar al atender, en $O(1)$, y desapilar al deshacer, en $O(1)$. Se implementó como lista enlazada con solo referencia al tope, ya que todas las operaciones ocurren en ese extremo. Consultar el historial completo (RF6) recorre la pila en $O(h)$, siendo h el número de atenciones.

En los algoritmos, $L[1..5]$ denota el arreglo de colas por nivel y S la pila del historial.

Algoritmo 2 Cancelar una solicitud pendiente

```
1: function CancelRequest(id)
2:     for u ← 1 to 5 do
3:         prev ← NULL; cur ← L[u].head
4:         while cur ≠ NULL do
5:             if cur.value.id = id then
6:                 if prev = NULL then
7:                     L[u].head ← cur.next
8:                 else
9:                     prev.next ← cur.next
10:                end if
11:                if cur = L[u].tail then
12:                    L[u].tail ← prev
13:                end if
14:                return TRUE
15:            end if
16:            prev ← cur; cur ← cur.next
17:        end while
18:    end for
19:    return FALSE
20: end function
```

Cuadro I
Estado de las funcionalidades en la primera entrega

Funcionalidad	Estado
Registrar solicitud (RF1)	Implementada
Atender por prioridad (RF2)	Implementada
Buscar por identificador (RF3)	Implementada
Cancelar solicitud (RF4)	Implementada
Deshacer última atención (RF5)	Pendiente
Consultar historial (RF6)	Implementada
Interfaz de consola	Implementada
Validación de entradas	Parcial
Pruebas del sistema	Parcial
Índice por identificador en $O(1)$	Pendiente
Análisis experimental de desempeño	Pendiente

IV. Estado de la implementación

La Tabla I resume el estado de cada funcionalidad a la fecha de esta entrega, distinguiendo entre funcionalidades implementadas, parcialmente implementadas y pendientes.

Implementadas. Cinco de los seis requerimientos funcionales están disponibles en la clase `TriageSystem`: `registerRequest`, `serveNext`, `findById`, `cancelRequest` y `getHistory`. La interfaz de consola (`Main`) expone todas estas operaciones mediante un menú. Las estructuras de soporte (`LinkedQueue`, `LevelPriorityQueue` y `LinkedStack`) son implementación propia; de la biblioteca estándar de Java solo se usa `ArrayList` para devolver el historial como lista.

Parcialmente implementadas. La validación de entradas rechaza identificadores vacíos o repetidos entre las solicitudes pendientes, urgencias fuera del rango 1–5 y urgencias no numéricas, pero aún no valida el nombre de la solicitud. Las pruebas se realizan mediante un programa

de prueba manual (`BasicTests.java`) que imprime el resultado esperado junto al obtenido, sin aserciones automáticas. El escenario principal registra cuatro solicitudes, verifica el orden de atención por urgencia y el desempate por orden de llegada entre dos solicitudes de urgencia 1, cancela una solicitud pendiente y comprueba que deja de encontrarse y de contarse, y confirma el orden del historial. Los casos borde cubren una urgencia fuera de rango, un identificador repetido, la cancelación de un identificador inexistente, la reutilización del identificador de una solicitud ya atendida y la atención con la cola vacía.

Pendientes. Para la siguiente entrega quedan deshacer la última atención (RF5), cuyo diseño se apoya en la pila ya implementada; un índice por identificador que reduzca la búsqueda y la cancelación a tiempo esperado constante; y la migración de las pruebas manuales a pruebas automatizadas. Para la entrega final queda un análisis experimental de tiempos de ejecución sobre volúmenes de datos mayores.

No se reportan resultados experimentales de desempeño en esta entrega, dado que el énfasis de la primera versión es la corrección funcional y la justificación del diseño; el análisis de complejidad teórica se presentó en la Sección III.

V. Discusión

El diseño adoptado separa la responsabilidad de cada estructura: la cola de prioridad por niveles decide el orden de atención, la lista enlazada de cada nivel permite retirar solicitudes canceladas, y la pila registra el historial. Esta separación evita resolver todas las necesidades con una sola estructura, por ejemplo una lista que se recorra linealmente para todo, lo que degradaría también la atención a $O(n)$.

La principal limitación conocida es el costo lineal de la búsqueda por identificador, que se traslada a la cancelación y a la verificación de identificadores repetidos al registrar. Para el tamaño de datos esperado (una fila de atención, no miles de solicitudes simultáneas) este costo se considera aceptable en esta etapa. Una mejora prevista es mantener un índice por identificador que permita ubicar cada solicitud en tiempo esperado constante sin alterar el orden de atención.

Otra limitación es que el costo constante de la cola de prioridad depende de que la urgencia tome solo cinco valores. Si en el futuro la prioridad dependiera de una magnitud continua, como el tiempo de espera acumulado, habría que evaluar otra representación de la cola de prioridad.

Finalmente, los datos existen solo durante la ejecución del programa, lo cual es consistente con el alcance definido (Sección III).

VI. Conclusiones

Esta primera entrega formuló el problema de priorizar y gestionar solicitudes de atención y presentó una solución basada en estructuras lineales: una cola de prioridad formada por una cola enlazada por nivel de urgencia, que registra y atiende en tiempo constante y resuelve el desempate por orden de llegada sin comparaciones adicionales, y una pila enlazada para el historial de atenciones. La implementación inicial cubre cinco de los seis requerimientos funcionales y fue verificada mediante un programa de prueba manual que recorre el escenario principal y varios casos borde.

Como trabajo futuro se plantea: implementar el deshacer de la última atención sobre la pila existente, incorporar un índice por identificador para reducir el costo de la búsqueda y la cancelación, añadir pruebas automatizadas y medir experimentalmente el desempeño con volúmenes de datos mayores.

Agradecimientos

Los autores agradecen a la Universidad Nacional de Colombia y al equipo docente de la asignatura Estructuras de Datos.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.